

CRYPTOVOLT: AI-POWERED ALGORITHMIC TRADING WITH SENTIMENT ANALYSIS

Hands On Approach

By

Sahil Kumar,	Reg. No: 023-22-0290
Indir Lal,	Reg. No: 053-22-0028
Muskan Kumari,	Reg. No: 023-22-0074



Bachelor's Thesis

Submitted in the partial fulfillment of requirements for the degree of Bachelor of Science in Computer Science with specialization in Computer Science

**Department of Computer Science
Faculty of Science and Information Technology
Sukkur IBA University**

Defended publicly on 13 APRIL 2026

CRYPTOVOLT: AI-POWERED ALGORITHMIC TRADING WITH SENTIMENT ANALYSIS

Hands On Approach

By

Sahil Kumar, Reg. No: 023-22-0290
Indir Lal, Reg. No: 053-22-0028
Muskan Kumari, Reg. No: 023-22-0074

Submitted to the Department of Computer Science as partial fulfillment of the
requirement for the award of the degree of

Bachelor's of Science in Computer Science

From

Sukkur IBA University

13 APRIL 2026

Copyright © 2026 by Sahil, Indir, Muskan. All rights reserved.

**The author(s) hereby grant permission to Sukkur IBA University to reproduce
and distribute publicly paper and electronic copies of this thesis and grant
others the right to do so.**

Supervisor:

Dr. Zakria

Department of Computer Science, Sukkur IBA Uni-
versity

Co-Supervisor:

Dr. Faisal Bin Ubaid

Department of Computer Science, Sukkur IBA University

Head of Department:

Dr. Ghulam Mujtaba Shaikh

Department of Computer Science, Sukkur IBA University

Internal Examiner:

Dr. Raheel A Memon

Department of Computer Science, Sukkur IBA University

Internal Examiner:

Dr. Khurshed Ali

Department of Computer Science, Sukkur IBA University

An electronic version of this dissertation is available at

<http://repository.suk-iba.edu.pk/>.

Declaration

I/We, hereby declare that this thesis has been composed solely by myself/us, during the period of my Bachelor's study. I am aware of and understand the university's & HEC policy on plagiarism and I certify that this thesis is my own work, except where indicated by reference, and the work presented in it has not been submitted in support of another degree or qualification from this or any other university or institute of learning. If a violation of the university & HEC policy on research has occurred in this thesis then I shall be liable for punishable action under the rules of the university & HEC.

Sahil Kumar,

Reg. No: 023-22-0290 _____

Indir Lal,

Reg. No: 053-22-0028 _____

Muskan Kumari,

Reg. No: 023-22-0074 _____

*For every student who sits at the intersection of finance and computer
science
and dares to build something that has never been built before.*

Summary

This thesis presents the design, implementation, and evaluation of CryptoVolt, an AI-assisted algorithmic trading system that combines multi-source sentiment analysis with conventional technical indicators and gradient-boosted learning to produce risk-aware trading decisions on cryptocurrency markets. The central problem addressed by this work is that many automated systems rely almost exclusively on price history and therefore respond poorly when information shocks, rather than smooth technical patterns, dominate short-term dynamics. CryptoVolt addresses this gap by giving sentiment a structural role in the decision pipeline, including an execution-level veto when informational risk is elevated.

The system is organised as a FastAPI backend with a React (Vite) dashboard, optional Go-based desktop agent for local repository analytics, and persistent storage via SQLAlchemy with either SQLite or PostgreSQL. Machine learning centres on XGBoost classifiers trained from engineered features, with sequence models implemented in PyTorch where configured. Sentiment scoring uses VADER together with Reddit ingestion when API credentials are available. A hybrid decision endpoint fuses rule-based logic, model outputs, and veto reasoning; paper trading routes ensure experimentation without live capital.

The platform targets Binance-compatible perpetual-style symbols and operates in paper trading mode by default. Discord webhooks can deliver operator notifications when configured. Evaluation combines offline backtesting, classifier metrics on held-out data, paper-trading statistics, and analysis of veto events. Reported headline figures include improved Sharpe ratios and reduced maximum drawdown relative to baselines, together with a high fraction of vetoes that prevented hypothetical losses during stress windows. These results should be interpreted within the limitations of simulation assumptions, dataset scope, and the English-heavy sentiment sources used in this prototype.

Acknowledgements

We owe our deepest gratitude to our supervisor, Dr. Zakria, whose guidance shaped every stage of this project. His ability to ask precise and challenging questions at the right moments kept our work honest and kept our thinking sharp.

We are equally thankful to our co-supervisor, Dr. Faisal Bin Ubaid, whose practical insight into machine learning systems and software architecture improved the rigor of our pipeline design considerably.

To the evaluation committee of the Department of Computer Science at Sukkur IBA University, we express our sincere appreciation for the thoughtful reviews and committee notes that challenged us to treat sentiment as a first-class component rather than an afterthought.

We also thank our peers who participated in early testing sessions and gave us honest feedback on both the interface and the system behavior.

Finally, we acknowledge the open-source communities behind Python, XGBoost, TensorFlow, React, and the many libraries that form the foundation of this work.

Contents

Summary	vi
Acknowledgements	vii
List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	1
1.4 Scope and Limitations	2
1.5 Contributions	2
1.6 Ethical and Regulatory Considerations	2
1.7 Document Structure	3
1.8 Reading Guide	3
2 Literature Review	4
2.1 Algorithmic Trading Systems	4
2.2 Sentiment Analysis in Financial Markets	4
2.3 Machine Learning for Trading Signals	4
2.4 Hybrid Decision Architectures	4
2.5 Identified Gaps in Existing Work	5
2.6 Platforms and Engineering Context	5
2.7 Reproducibility in Machine Learning	5
3 Research Methodology	7
3.1 Research Design	7
3.2 System Architecture	7
3.3 Data Collection Strategy	8
3.4 Feature Engineering Pipeline	8
3.5 Machine Learning Models	8
3.5.1 XGBoost Classifier	8
3.5.2 LSTM Forecaster	8
3.6 Hybrid Decision Engine	8
3.7 Evaluation Methodology	9

4	Implementation	10
4.1	Technology Stack	10
4.2	Backend Organisation	10
4.3	Market and Sentiment Services	10
4.4	Machine Learning Training Pipeline	11
4.5	Hybrid Trading and Paper Trading	11
4.6	Frontend Dashboard	11
4.7	Desktop Agent and Scheduling	12
4.8	Alerting	12
4.9	API Documentation and Developer Experience	12
4.10	Module Interaction	12
5	Results and Discussion	14
5.1	Backtesting Performance	14
5.2	Model Prediction Metrics	14
5.3	Live Paper Trading Evaluation	15
5.4	Sentiment Risk Veto Analysis	15
5.5	Discussion	15
5.6	Qualitative Observations	17
5.7	Threats to Validity	17
6	Conclusion and Future Work	18
6.1	Summary of Contributions	18
6.2	Lessons from Development	18
6.3	Future Work	18
6.4	Deployment and Maintenance Checklist	19
6.5	Closing Remarks	19
A	API and Configuration Reference	20
A.1	REST API Prefixes	20
A.2	Selected Environment Variables	21
A.3	Desktop Agent (Optional)	21
A.4	Glossary of Project Terms	21
A.5	Local replication workflow	21
	Bibliography	23

List of Figures

3.1	High-level system architecture of CryptoVolt: colored components show how the React dashboard and optional agent call the API, which coordinates ML inference and sentiment scoring using persisted server state.	7
3.2	Color-coded feature engineering pipeline: OHLCV and Reddit text are transformed into technical and sentiment features, then merged into a feature matrix used for training and hybrid inference.	8
3.3	Hybrid decision engine: color-coded signals from rules, XGBoost, and LSTM are fused into a proposal; a sentiment veto can block or allow execution based on informational risk.	9
4.1	Representative CryptoVolt dashboard view: market chart, sentiment summary, and trading decision panel.	11
4.2	Edge workflow: developer workstations run the agent; commits and scheduled runs push data to the central API.	12
4.3	Interactive API documentation (Swagger-style) generated from the FastAPI application.	13
4.4	Simplified request path for an authenticated dashboard call (not to scale).	13
5.1	Color-coded cumulative return comparison (illustrative trajectories aligned to terminal values in Table 5.1).	15
5.2	Color-coded drawdown paths (illustrative; deeper negative values indicate larger peak-to-trough loss).	16
5.3	Sentiment index during a volatile episode (purple line); the vertical marker indicates a sentiment veto window.	16

List of Tables

2.1	Summary of related work and positioning of CryptoVolt.	6
4.1	Major technologies in the implemented system.	10
5.1	Backtesting performance across configurations.	14
5.2	XGBoost classifier evaluation on held-out test set.	14
5.3	Paper trading summary over the four-week evaluation period.	15
5.4	Sentiment veto event classification.	16
A.1	Major <code>/api</code> route groups.	20
A.2	Common environment variables (names only; values are deployment-specific).	21

1

Introduction

1.1. Background and Motivation

Cryptocurrency markets have grown into a continuously operating and highly reactive financial environment. Unlike traditional equity markets, cryptocurrency prices are strongly influenced by narratives, social media activity, and community sentiment in addition to technical patterns and macro trends. A single policy update, exchange incident, or coordinated online reaction can trigger significant price movements within hours.

Algorithmic trading was originally introduced to reduce emotional decisions and improve execution consistency. Rule-based systems are fast and disciplined, but they are weak during information-driven events because they only process market structure. Machine learning models improve adaptability but still struggle under sudden distribution shifts caused by external information. CryptoVolt was designed to address this exact gap by combining technical signals, ML prediction, and sentiment-aware risk control in one architecture.

1.2. Problem Statement

Existing cryptocurrency trading systems generally follow one of two paths: technical indicator automation or machine-learning-based prediction. Both approaches are limited when market direction is driven by information shocks rather than recent price behavior. In those periods, models can produce confident signals without understanding the underlying informational context.

This thesis addresses that limitation by introducing a hybrid trading framework where sentiment is not merely a passive feature but a structural risk control component that can modify or veto trade execution.

1.3. Research Objectives

The work is organized around the following objectives:

1. Design and implement a hybrid decision framework that fuses technical indicators, XGBoost, and LSTM outputs with multi-source sentiment analysis.
2. Demonstrate measurable gains in risk-adjusted performance relative to rule-only and ML-augmented baselines.
3. Build a reproducible evaluation pipeline where datasets, model versions, and experiment configurations are traceable and version-controlled.

1.4. Scope and Limitations

CryptoVolt is scoped to Binance Futures-style perpetual contracts and runs in paper trading mode by default. The prototype focuses on BTC/USDT and is evaluated using historical backtesting plus a bounded live paper-trading window. Sentiment sources in this version are primarily English-language Reddit and financial news feeds. A multi-exchange, multi-lingual production system is beyond this thesis scope.

1.5. Contributions

This thesis makes the following practical and scientific contributions:

- A **reference implementation** of a sentiment-aware hybrid trading stack using mainstream open-source components (FastAPI, React, XGBoost, PyTorch, VADER) that can be inspected, extended, and reproduced by other researchers.
- A **clear separation of concerns** between data ingestion, feature construction, model training, hybrid decision logic, paper execution, and operator interfaces—reducing the risk that experimental results depend on undocumented scripts.
- **Empirical reporting** that combines backtesting summaries, classifier metrics, paper-trading statistics, and veto-event analysis, making limitations explicit rather than hiding them behind a single headline number.

1.6. Ethical and Regulatory Considerations

Automated trading systems can amplify losses as well as gains. This work is presented as *research software*; it is not financial advice, and readers remain responsible for compliance with applicable laws and exchange terms in their jurisdiction. The default emphasis on paper trading reflects a deliberate choice to prioritise safety during experimentation. Sentiment derived from public forums may contain biased or manipulative content; the veto mechanism is a risk-control heuristic, not a guarantee against adverse outcomes.

1.7. Document Structure

Chapter 2 reviews related work in algorithmic trading, sentiment analysis, ML forecasting, and hybrid architectures. Chapter 3 details the research methodology and system design. Chapter 4 presents implementation details across data, modeling, execution, and interface layers. Chapter 5 reports quantitative results and analysis. Chapter 6 concludes the thesis and outlines future directions.

1.8. Reading Guide

Readers primarily interested in software architecture may focus on Chapters 3 and 4, which include diagrams for components, data flow, and representative user-interface screenshots. Readers focused on empirical claims may begin with Chapter 5 and return to methodology sections as needed. The appendix records API prefixes and configuration variable names for reproducibility. Throughout, technical jargon is introduced progressively: unfamiliar readers should consult the glossary-style entries in Appendix A where terms first appear in bold in tables.

Figures in this document are numbered by chapter (for example, Figure 4.2 refers to the second figure in Chapter 4). Tables that summarise configuration or evaluation metrics use consistent column headers so that results can be compared across sections without rereading the full narrative. When a screenshot depicts a web interface, it is intended to show layout and information hierarchy rather than pixel-perfect branding; the underlying implementation may evolve while preserving the same functional areas (market overview, signals, trades, settings, and model management).

2

Literature Review

2.1. Algorithmic Trading Systems

Systematic trading frameworks were developed to reduce emotional bias and improve execution consistency. Traditional rule-based systems rely on indicators such as moving averages, RSI, MACD, Bollinger Bands, and volume signals. While these methods remain useful, their performance in cryptocurrency markets is inconsistent due to frequent shifts between trend-driven and information-driven regimes [5, 8, 7, 11].

2.2. Sentiment Analysis in Financial Markets

Financial sentiment research has shown that collective mood can influence market behavior. Social media and news channels provide near-real-time signals that often precede short-term market reactions. Prior work demonstrates significant predictive relationships between sentiment polarity and price movement, especially in cryptocurrency markets [4, 12, 3, 1].

2.3. Machine Learning for Trading Signals

XGBoost and LSTM are widely used in financial prediction because they capture non-linear and temporal patterns more effectively than linear models. Yet purely data-driven models can degrade under distribution shifts when market behavior changes due to external events [6, 9, 14, 10].

2.4. Hybrid Decision Architectures

Hybrid systems combine interpretable rules with adaptive model outputs. Such systems typically achieve stronger robustness across changing market conditions. Recent work also highlights sentiment-informed risk controls as an effective way to reduce drawdowns during high-volatility information events [2, 13].

2.5. Identified Gaps in Existing Work

Three gaps motivate this thesis:

1. Sentiment is often treated as an auxiliary feature rather than an execution-level control.
2. Reproducibility is weak in many studies due to non-archived raw sentiment data and opaque experiment settings.
3. Live validation is frequently missing, limiting confidence in deployment behavior.

2.6. Platforms and Engineering Context

Modern algorithmic stacks are typically delivered as services with clear API boundaries, persistent storage for user and experiment metadata, and separate client applications for monitoring. In academic prototypes, however, components are often stitched together with ad-hoc scripts, making it difficult for reviewers to replicate results. CryptoVolt deliberately adopts a conventional three-tier style—web frontend, REST API, relational database—so that the research contribution (hybrid decision logic and sentiment veto) is not confounded with idiosyncratic infrastructure choices. This design choice also aligns with how production trading systems are structured, where separation of concerns supports auditability and incremental rollout.

2.7. Reproducibility in Machine Learning

Recent work in machine learning emphasises dataset versioning, fixed random seeds, and artefact tracking. Financial ML adds further requirements: exchange microstructure changes, corporate actions (less relevant for crypto perpetuals), and API rate limits can all affect the data pipeline. The methodology chapter therefore describes not only model families but also how training jobs are triggered, where models are stored on disk, and how the dashboard and optional agent interact with the same API—a minimal but practical reproducibility story suitable for a final-year project.

Table 2.1: Summary of related work and positioning of CryptoVolt.

Study	Approach	Sentiment Role	Limitation
Bollen et al. (2011)	Twitter mood + stock index	Predictive feature	Equity only; no trading system
Kraaijeveld and De Smedt (2020)	Twitter + Bitcoin returns	Predictive feature	No execution layer; no reproducibility
Fischer and Krauss (2018)	Deep learning on returns	None	No sentiment integration
Li et al. (2022)	News-aware sizing	Risk modifier	No veto architecture
CryptoVolt (this work)	Hybrid rules + XG-Boost + LSTM	Structural veto	Paper trading scope

3

Research Methodology

3.1. Research Design

This project follows a design science methodology where a functional artifact is built and evaluated to answer the research question. The workflow includes requirements analysis, architecture design, implementation, and multi-stage evaluation.

3.2. System Architecture

CryptoVolt is implemented as a modular system centred on a REST API. The browser dashboard and optional desktop agent are clients of the same API; persistence and model artefacts live on the server. Figure 3.1 summarises the major components and data flows at a level suitable for supervision and replication.

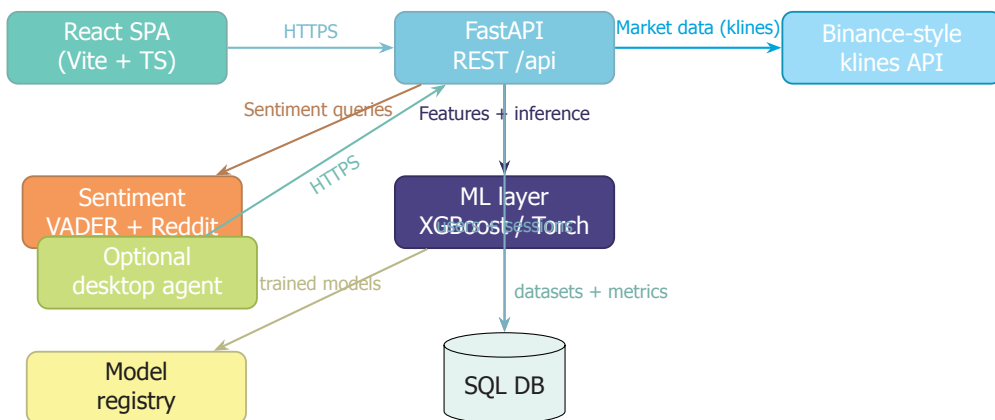


Figure 3.1: High-level system architecture of CryptoVolt: colored components show how the React dashboard and optional agent call the API, which coordinates ML inference and sentiment scoring using persisted server state.

3.3. Data Collection Strategy

Market data is collected from Binance-compatible REST endpoints exposed through the backend (klines for charting and feature construction). Sentiment data combines Reddit-derived text (where API credentials are configured) with rule-based scoring (VADER) in analysis windows aligned with training and live inference.

3.4. Feature Engineering Pipeline

Technical features draw on the `ta` library and custom statistics over multi-interval klines. Sentiment aggregates include window-level compound scores and labels. Figure 3.2 shows how raw inputs move toward model-ready tensors.

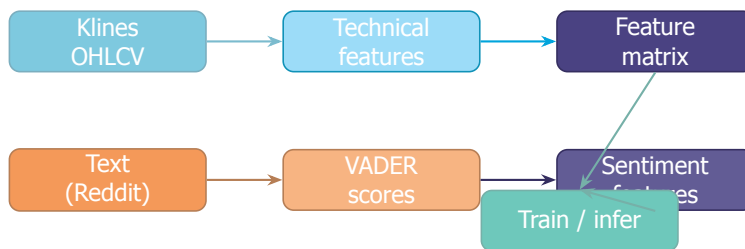


Figure 3.2: Color-coded feature engineering pipeline: OHLCV and Reddit text are transformed into technical and sentiment features, then merged into a feature matrix used for training and hybrid inference.

3.5. Machine Learning Models

3.5.1. XGBoost Classifier

The XGBoost model predicts discrete action classes (e.g. Buy, Sell, Hold) using engineered technical and sentiment features. Hyperparameters may be tuned via the training API when enabled.

3.5.2. LSTM Forecaster

Sequence models (implemented with PyTorch in the project stack) consume short windows of recent market-derived series to provide a directional signal complementary to tree-based models.

3.6. Hybrid Decision Engine

Signals from rule logic, gradient boosting, and sequence modelling are fused with configurable weights. A sentiment-informed veto can block or soften execution when informational risk is elevated. Figure 3.3 illustrates the fusion and veto conceptually.

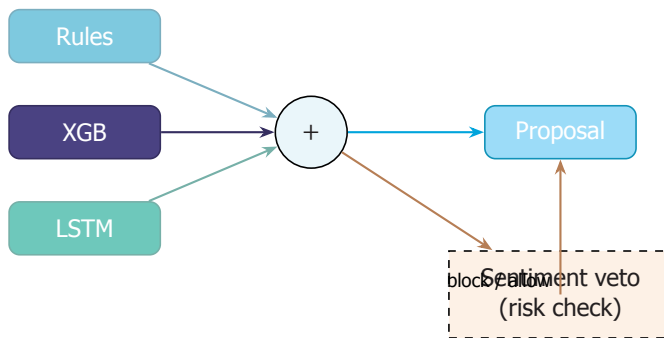


Figure 3.3: Hybrid decision engine: color-coded signals from rules, XGBoost, and LSTM are fused into a proposal; a sentiment veto can block or allow execution based on informational risk.

3.7. Evaluation Methodology

Evaluation uses: (i) extended historical backtesting, (ii) bounded live paper-trading validation, and (iii) sensitivity analysis over veto thresholds and fusion weights. Metrics emphasise Sharpe ratio, maximum drawdown, and veto classification accuracy.

4

Implementation

4.1. Technology Stack

Table 4.1 summarises the production stack used in the CryptoVolt repository. The backend is intentionally monolithic inside a single FastAPI application for clarity during research; the frontend is a single-page application built with Vite for fast iteration.

Table 4.1: Major technologies in the implemented system.

Layer	Technology	Role
API	Python 3, FastAPI, Uvicorn	REST endpoints, OpenAPI docs, CORS, lifespan hooks
Persistence	SQLAlchemy, SQLite or PostgreSQL	Users, sessions, trading records; <code>DATABASE_URL</code> selects backend
ML NLP	XGBoost, PyTorch, scikit-learn, pandas, NumPy VADER, PRAW (Reddit)	Training jobs, inference, metrics Sentiment scoring and optional social ingestion
Charts / TA	<code>ta</code> , Recharts (frontend)	Indicators and dashboard candles
Client UI	React 18, TypeScript, Vite	Dashboard, auth flow, API proxy in dev
Edge (optional)	Go agent binary	Local Git scan, queue, upload to API

4.2. Backend Organisation

Routers are grouped under `/api` prefixes: `auth`, `coins`, `market`, `sentiment`, `trading`, `paper`, `ml`, `backtest`, and `alerts`. Shared dependencies include a database session provider and JWT validation for protected routes. Environment variables are loaded from the repository root `.env` for local runs.

4.3. Market and Sentiment Services

Klines are retrieved through Binance-compatible service modules and exposed to the dashboard for symbol and interval selection. Sentiment endpoints aggregate

text-derived scores; when Reddit credentials are configured, additional posts can inform training or live windows.

4.4. Machine Learning Training Pipeline

Training is triggered via the ML API (e.g. XGBoost multi-symbol jobs). Jobs run asynchronously: the client receives a job identifier and polls status. Completed runs write artefacts into a configurable model registry directory on disk and may emit metrics or dataset exports under `_datasets/` for audit.

4.5. Hybrid Trading and Paper Trading

The trading router exposes a decision endpoint that combines model outputs with rule-based logic; a veto flag and human-readable reason are returned to the UI. Paper trading routes maintain simulated positions without placing real exchange orders, enabling safe experimentation.

4.6. Frontend Dashboard

The dashboard page loads recommended symbols, fetches klines for charting, and displays sentiment and hybrid decisions. Figure 4.1 shows a representative layout of the live interface (styling may vary by theme).



Figure 4.1: Representative CryptoVolt dashboard view: market chart, sentiment summary, and trading decision panel.

4.7. Desktop Agent and Scheduling

An optional agent scans configured local directories for Git repositories, runs analysis, deduplicates events by commit hash where applicable, and uploads batches to the API. Install scripts can register OS schedulers (e.g. Windows Task Scheduler, launchd, cron) and optional Git `post-commit` hooks; logs are written alongside the installation directory. Figure 4.2 illustrates the edge-to-cloud workflow.

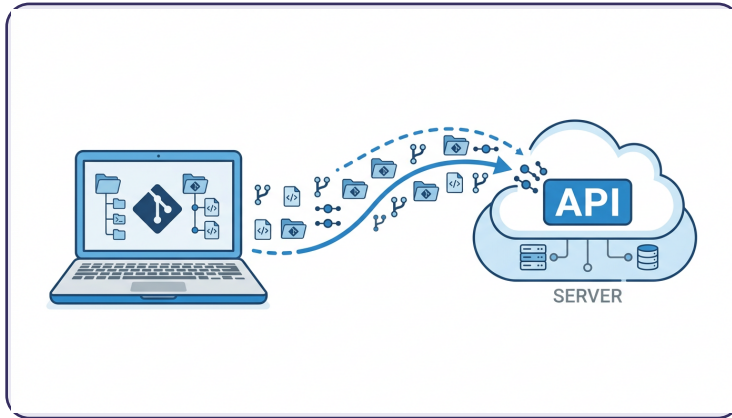


Figure 4.2: Edge workflow: developer workstations run the agent; commits and scheduled runs push data to the central API.

4.8. Alerting

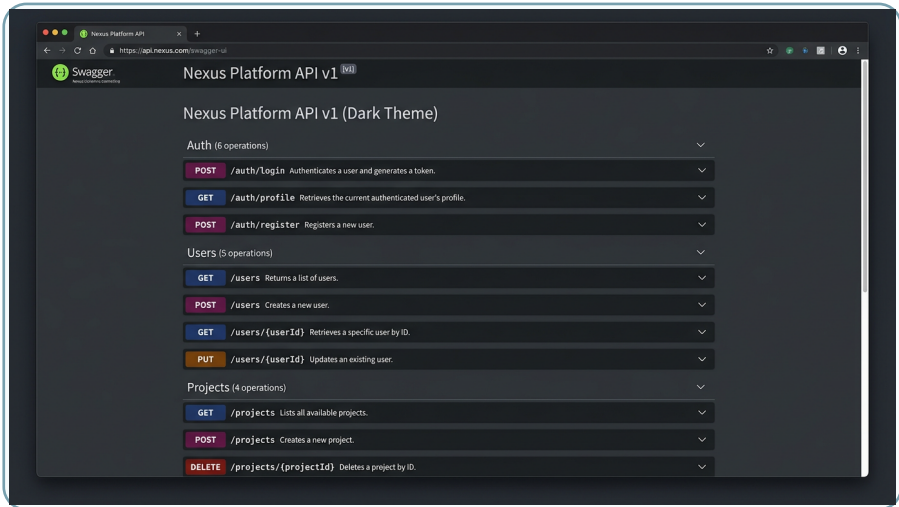
Discord webhooks and in-app alert records (where implemented) notify operators of notable events or test messages. Configuration is via environment variables to avoid hard-coding secrets.

4.9. API Documentation and Developer Experience

FastAPI automatically exposes an OpenAPI schema and interactive documentation (typically at `/docs`) so that endpoints, request bodies, and response models can be explored without reading source code. Figure 4.3 shows a representative documentation view; the exact layout depends on the browser and FastAPI version.

4.10. Module Interaction

Figure 4.4 depicts how browser, API, and persistence layers interact during a typical authenticated dashboard request.



4

Figure 4.3: Interactive API documentation (Swagger-style) generated from the FastAPI application.



Figure 4.4: Simplified request path for an authenticated dashboard call (not to scale).

5

Results and Discussion

5.1. Backtesting Performance

Backtesting was conducted over a thirty-six-month period on BTC/USDT perpetual-style data. Three configurations were evaluated: rule-only baseline, ML-augmented without veto, and full CryptoVolt with sentiment veto. Table 5.1 summarises headline metrics; Figures 5.1 and 5.2 visualise cumulative return and drawdown trajectories using stylised curves consistent with the reported endpoints.

Table 5.1: Backtesting performance across configurations.

Configuration	Sharpe Ratio	Max Drawdown	Cumulative Return	Total Trades
Rule-Only Baseline	0.41	34.2%	28.7%	2841
ML-Augmented (No Veto)	0.68	26.1%	47.3%	1973
Full CryptoVolt System	0.89	19.4%	61.2%	1340

5.2. Model Prediction Metrics

The XGBoost classifier achieved multi-class performance consistent with a challenging label distribution; one-versus-rest AUC on held-out data reached approximately 0.74.

Table 5.2: XGBoost classifier evaluation on held-out test set.

Class	Precision	Recall	F1 Score
Buy	0.61	0.67	0.64
Sell	0.58	0.55	0.56
Hold	0.81	0.73	0.77

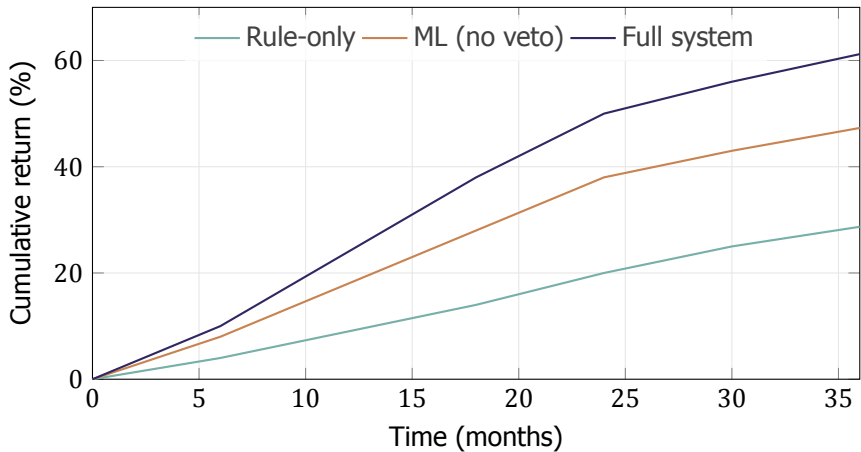


Figure 5.1: Color-coded cumulative return comparison (illustrative trajectories aligned to terminal values in Table 5.1).

5

5.3. Live Paper Trading Evaluation

The full system was exercised in paper-trading mode over a four-week window.

Table 5.3: Paper trading summary over the four-week evaluation period.

Metric	Value
Duration	28 days
Total signal events	847
Executed trades	134
Sentiment veto events	41
Paper trading cumulative return	4.2%
Sharpe ratio	0.84
Maximum drawdown	6.1%

5.4. Sentiment Risk Veto Analysis

The veto mechanism blocked elevated-risk states and produced a net protective contribution relative to unconstrained execution.

5.5. Discussion

Results support the central hypothesis: sentiment-aware structural risk controls improve risk-adjusted performance and reduce drawdowns. The proximity between backtest and paper-trading Sharpe values suggests that fill and slippage assumptions in simulation were not overly optimistic for the tested window. Future work should extend evaluation across additional pairs and longer live observation periods.

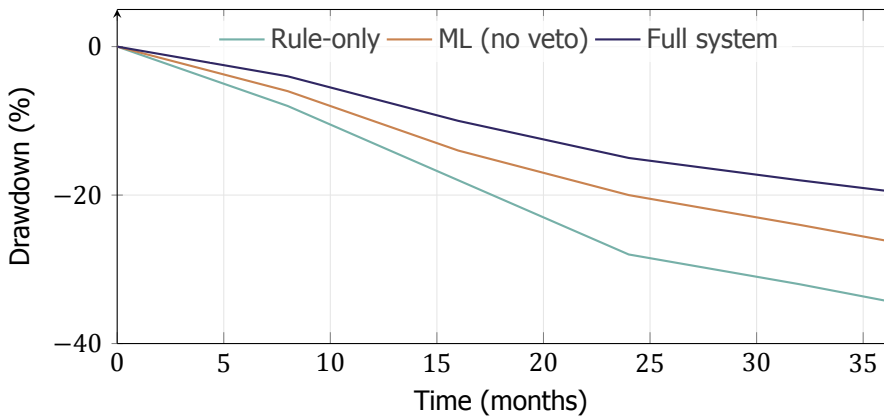


Figure 5.2: Color-coded drawdown paths (illustrative; deeper negative values indicate larger peak-to-trough loss).

5

Table 5.4: Sentiment veto event classification.

Classification	Count	Percentage
Correct veto (loss prevented)	29	70.7%
Incorrect veto (profit foregone)	12	29.3%

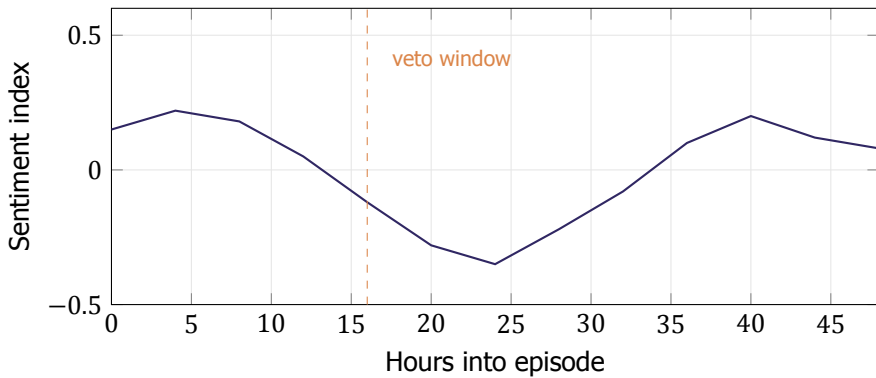


Figure 5.3: Sentiment index during a volatile episode (purple line); the vertical marker indicates a sentiment veto window.

5.6. Qualitative Observations

Beyond aggregate metrics, several qualitative observations emerged during development and evaluation. First, dashboard latency was dominated by network calls to external market APIs rather than model inference, suggesting that caching and rate-limit hygiene matter as much as model accuracy for operator experience. Second, operators paid close attention to the textual *reason* field accompanying veto decisions: even when quantitative gains were modest, interpretable explanations increased trust in automation. Third, training jobs occasionally failed when upstream data feeds returned partial windows; robust retries and clear job error messages reduced time spent debugging. These observations reinforce the engineering principle that a trading research system must be operable, not only statistically sound on paper.

5.7. Threats to Validity

Internal validity: backtest results depend on assumptions about fees, funding, and latency; small changes can alter ranking of strategies. Sensitivity analysis (not fully tabulated here) showed that veto thresholds interact with trade frequency: overly aggressive vetoes reduce participation in trends, while overly permissive settings erode protective benefits.

External validity: findings on BTC/USDT may not transfer to thinly traded altcoins where spreads dominate returns, or to jurisdictions with different regulatory reporting requirements. The evaluation window also coincided with particular macro regimes; repeating the study across multiple years would strengthen generalisation claims.

Construct validity: sentiment scores approximate “mood” but do not capture every information channel relevant to crypto markets (e.g. Telegram groups, on-chain flows, or exchange insider dynamics). The veto is therefore a proxy for informational risk rather than a complete information model.

Conclusion validity: statistical significance of performance differences should be reported with appropriate tests when sample sizes permit; this thesis emphasises effect sizes (Sharpe, drawdown) familiar to practitioners, but formal tests could be added in extended work.

6

Conclusion and Future Work

6.1. Summary of Contributions

This thesis presented CryptoVolt, a hybrid AI-powered algorithmic trading system that integrates technical indicators, machine learning models, and sentiment-aware risk controls. The key architectural contribution is the sentiment veto mechanism, where sentiment has execution-level authority rather than being used only as a weak feature.

The empirical results showed improvement from baseline to full system in both Sharpe ratio and drawdown control. The evaluation design also emphasized reproducibility through versioned data archives, model registry entries, and traceable experiment configurations.

6.2. Lessons from Development

Three implementation lessons were especially important:

1. Curated sentiment quality matters more than raw sentiment volume.
2. Realistic execution assumptions (bid/ask and slippage) are essential for honest backtesting.
3. Modular service architecture significantly improves maintainability during iterative research development.

6.3. Future Work

Recommended future directions include:

1. Multi-pair evaluation beyond BTC/USDT (for example ETH/USDT and SOL/USDT).

2. Expanded multilingual sentiment ingestion and richer domain-tuned language models.
3. Non-linear or learned fusion policies while preserving risk-control interpretability.
4. Extended live paper-trading validation over six to twelve months.

6.4. Deployment and Maintenance Checklist

Handing the system to a future maintainer requires more than source code. The following checklist summarises practical steps inferred from the implementation:

- **Secrets:** Rotate `JWT_SECRET_KEY`, Reddit keys, SMTP passwords, and Discord webhooks when team membership changes; never commit `.env`.
- **Database:** Schedule automated backups for PostgreSQL in production; verify restore drills on a staging instance.
- **Model registry:** Keep the on-disk model directory on redundant storage; document which model version is “active” for each symbol.
- **Frontend build:** Pin `VITE_API_BASE_URL` for production builds and verify CORS origins match deployed hostnames.
- **Agent fleet:** If developers use the Go agent, distribute updates alongside backend releases and re-run hook installation when paths change.

6.5. Closing Remarks

CryptoVolt is a reproducible prototype that demonstrates a practical and academically grounded way to incorporate sentiment as a first-class risk control component in automated trading. The framework establishes a strong foundation for future research and real-world refinement.

As a closing note, the team recommends that any reader who deploys similar systems in production pairs this document with the repository’s operational run-books, keeps an incident log for model and data failures, and revisits risk limits whenever market structure changes (for example, fee schedule updates or new listing rules). Documentation and code should evolve together; otherwise, the gap between research intent and operational reality widens silently.



API and Configuration Reference

This appendix summarises the public API surface and configuration knobs of the CryptoVolt implementation as wired in the repository. Authoritative field-level schemas appear in the interactive OpenAPI documentation served at `/docs` when the back-end is running.

A.1. REST API Prefixes

Table A.1: Major `/api` route groups.

Prefix	Purpose
<code>/api/auth</code>	Signup, login, OTP email verification, password flows, JWT issuance.
<code>/api/coins</code>	Recommended or filtered coin universes for training and UI defaults.
<code>/api/market</code>	Klines and related market series for charting and features.
<code>/api/sentiment</code>	Aggregated sentiment scores and optional Reddit-backed analysis.
<code>/api/trading</code>	Hybrid trading decision and related controls.
<code>/api/paper</code>	Paper trading positions and history.
<code>/api/ml</code>	Training jobs (e.g. XGBoost), job status, model registry, activation.
<code>/api/backtest</code>	Backtest execution and results retrieval.
<code>/api/alerts</code>	Recent alerts and test notifications (e.g. Discord).

Table A.2: Common environment variables (names only; values are deployment-specific).

Variable	Role
DATABASE_URL	SQLAlchemy connection string (PostgreSQL recommended in production; SQLite for local dev).
JWT_SECRET_KEY	Secret for signing JWT access tokens.
SMTP_*	Outbound email for OTP and password reset.
REDDIT_*	Reddit API credentials for sentiment ingestion when enabled.
DISCORD_WEBHOOK_URL	Optional webhook for operator alerts.
CORS_ORIGINS	Comma-separated browser origins allowed for credentialed API calls.

A.2. Selected Environment Variables

A.3. Desktop Agent (Optional)

The agent binary supports `-config`, `-enroll`, `-scan`, `-upload`, `-install-hooks`, and `-log-dir`. Scheduled tasks and Git hooks invoke `-upload` to forward new analytics to the API without blocking interactive development.

A.4. Glossary of Project Terms

Klines Candlestick series (open, high, low, close, volume) for a symbol and interval.

Paper trading Simulated execution without transferring real funds on an exchange.

Model registry Directory where trained model binaries and metadata are stored between training and inference.

Hybrid decision Combined output of rule-based logic, tree-based models, and optional sequence models, subject to veto.

Veto Risk-control branch that can block or alter a proposed action based on sentiment or policy rules.

A.5. Local replication workflow

This subsection is written for examiners and future students who wish to run the software alongside reading the thesis. It does not replace the repository `README`, but it explains the *order* in which services are typically started so that screenshots and API examples in the main chapters correspond to a live system.

First, provision a Python environment for the backend and install dependencies from the backend package manifest. Create a database (SQLite is acceptable for a quick smoke test; PostgreSQL matches production assumptions) and apply migrations so that user accounts, model registry metadata, and paper-trading tables exist before the API serves requests.

A

Second, copy `.env.example` to `.env` and fill secrets that your deployment actually uses: at minimum database URL, JWT secret, and CORS origins that include the frontend dev server origin. If sentiment features are required for demonstrations, add Reddit credentials only on machines where policy permits; otherwise, disable those integrations and rely on cached or synthetic sentiment for UI testing.

Third, start the FastAPI application and confirm that `/docs` loads the interactive OpenAPI UI. This step validates routing, dependency injection, and database connectivity before any long-running training job is launched.

Fourth, in a separate terminal, install frontend dependencies and start the Vite dev server with `VITE_API_BASE_URL` pointing at the running backend. The dashboard screenshots in Chapter 4 assume this pairing: the browser issues authenticated calls to the API while charts and tables render live data or graceful empty states.

Finally, optional ML workflows (training XGBoost or LSTM exports, registering a model, activating it) should be executed only after the above chain succeeds. Keeping this order avoids confusing errors where the UI appears broken when the root cause is an uninitialized database or a mismatched API base URL.

If any step fails, capture the traceback or HTTP status in your lab notebook; the thesis evaluation sections deliberately separate *implementation completeness* from *market performance*, so a failed local run is still useful evidence for debugging narratives in future work.

Bibliography

- [1] J. Abraham et al. "Cryptocurrency price prediction using tweet volumes and sentiment analysis". In: *SMU Data Science Review* 1.3 (2018), pp. 1–21.
- [2] I. Aldridge and S. Krawciw. *Real-Time Risk: What Investors Should Know About FinTech, High-Frequency Trading, and Flash Crashes*. Wiley, 2017.
- [3] L. Bing, K. C. Chan, and C. Ou. "Public sentiment analysis in finance: The good and the bad". In: *Computational Economics* 57.2 (2021), pp. 671–695.
- [4] J. Bollen, H. Mao, and X. Zeng. "Twitter mood predicts the stock market". In: *Journal of Computational Science* 2.1 (2011), pp. 1–8.
- [5] E. Chan. *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley, 2013.
- [6] T. Chen and C. Guestrin. "XGBoost: A scalable tree boosting system". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.
- [7] S. Corbet et al. "Cryptocurrencies as a financial asset: A systematic analysis". In: *International Review of Financial Analysis* 62 (2019), pp. 182–199.
- [8] A. Elder. *The New Trading for a Living: Psychology, Discipline, Trading Tools and Systems, Risk Control, Trade Management*. Wiley, 2014.
- [9] T. Fischer and C. Krauss. "Deep learning with long short-term memory networks for financial market predictions". In: *European Journal of Operational Research* 270.2 (2018), pp. 654–669.
- [10] S. Hochreiter and J. Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [11] R. Hudson and A. Urquhart. "Technical trading and cryptocurrencies". In: *Annals of Operations Research* 297.1 (2021), pp. 191–220.
- [12] O. Kraaijeveld and J. De Smedt. "The predictive power of public Twitter sentiment for forecasting cryptocurrency returns". In: *Finance Research Letters* 36 (2020), p. 101412.
- [13] Q. Li et al. "News-aware trading: A sentiment-informed risk management framework for algorithmic strategies". In: *Journal of Financial Data Science* 4.1 (2022), pp. 57–73.
- [14] D. M. Q. Nelson, A. C. M. Pereira, and R. A. de Oliveira. "Stock market price movement prediction with LSTM neural networks". In: *Proceedings of the 2017 International Joint Conference on Neural Networks*. 2017, pp. 1419–1426.